Evaluación experimental de YOLOv8, YOLO26 y familias R-CNN para detección y segmentación de lomos de libros en imágenes

Facundo Silberstein
Buenos Aires, Argentina
facundonsr@gmail.com

Francisco Meaca
Buenos Aires, Argentina
franmeaca99@gmail.com

Ian Sztenberg
Buenos Aires, Argentina
ianezequiel@gmail.com

Marcelo Luna
Buenos Aires, Argentina
mhluna@gmail.com

***Abstract*—Este trabajo evalúa la localización de lomos en imágenes de estanterías y la recuperación de texto mediante OCR, tomando como referencia el conjunto *Book Spine 2* (v4). La comparación abarca seis modelos agrupados por representación: YOLOv8s y Faster R-CNN (cajas alineadas a los ejes, AABB), YOLOv8s-OBB y YOLO26s-OBB (cajas orientadas, OBB), y YOLO26s-seg y Mask R-CNN (máscaras de instancia). Se analizan *mAP*, precisión, recall y latencia por tipo de tarea, se evalúa OCR sobre lomos localizados y se implementa un prototipo demostrativo del pipeline elegido. Los resultados muestran compromisos entre calidad de localización, recall en escenas densas o inclinadas y velocidad de inferencia, y sostienen un flujo basado en YOLO26s-seg y PaddleOCR.**

Keywords—detección de lomos, cajas orientadas, segmentación de instancias, OCR, YOLOv8, YOLO26, Faster R-CNN, Mask R-CNN, inventario de libros

I. Introducción

El control de inventario en bibliotecas y librerías depende con frecuencia de inspecciones visuales por estantería. Automatizar ese proceso a partir de fotografías exige, como mínimo, localizar cada lomo y recuperar el texto visible para contrastarlo con un catálogo. Las cajas alineadas a los ejes (AABB) son un punto de partida habitual; sin embargo, trabajos recientes muestran que degradan la localización en lomos inclinados [1], y el análisis exploratorio de *Book Spine 2* confirma además solapes densos y AABB holgados respecto del polígono anotado. Ese diagnóstico motiva ampliar la comparación a cajas orientadas (OBB) y a máscaras de instancia, en la línea de pipelines que combinan detección y OCR sobre lomos [2].

En este trabajo se comparan seis modelos —YOLOv8s y Faster R-CNN (AABB), YOLOv8s-OBB y YOLO26s-OBB (OBB), YOLO26s-seg y Mask R-CNN (máscaras)— y se evalúa OCR sobre los lomos localizados. La contribución es una comparación experimental acotada: mismo conjunto de datos, mismos criterios de informe por tipo de tarea, y foco en el compromiso entre calidad de localización, comportamiento en escenas densas o inclinadas, latencia y utilidad del recorte para OCR.

El resto del artículo organiza el argumento en ese orden: conjunto de datos y protocolo (Secciones III–IV), resultados de localización (Sección V), discusión y elección del localizador (Sección VI), experimentación con OCR (Sección VII), prototipo de demostración (Sección VIII) y conclusiones con trabajo futuro (Sección IX).

II. Trabajo relacionado

A. Inventario visual de lomos

La detección automática de lomos se ha planteado como alternativa a inventarios manuales y a sistemas RFID de elevado costo. Ma *et al.* [1] atacan el caso de lomos inclinados y de aspecto extremo con un Oriented R-CNN mejorado, y muestran que las cajas alineadas a los ejes degradan la localización cuando el libro deja de estar vertical. En una línea más cercana a un pipeline completo, Li *et al.* [2] combinan detección y segmentación basadas en YOLO con OCR (PaddleOCR) para recuperar texto en estanterías densas. A mayor escala, Llabrés *et al.* [3] formulan el inventario como emparejamiento *many-to-many* entre detecciones y un catálogo bajo OCR ruidoso. Ese problema excede el matching por similitud textual simple; aquí se menciona solo como horizonte, no como método adoptado.

B. Detección: one-stage, two-stage y orientación

Las arquitecturas *one-stage* de la familia YOLO priorizan velocidad e inferencia densa. Terven y Córdova-Esparza [5] revisan esa evolución hasta YOLOv8 y YOLO-NAS, mientras que Yaseen [6] detalla la arquitectura de YOLOv8 (backbone tipo CSP, cuello FPN/PAN, cabeza *anchor-free*). En el extremo *two-stage*, Faster R-CNN [4] unifica propuestas (RPN) y detección compartiendo cómputo convolucional, con el clásico intercambio entre mayor costo y propuestas explícitas. El lenguaje de evaluación —AP/*mAP* sobre umbrales de IoU— está consolidado por MS COCO [9] y es el que también se aplica sobre *Book Spine 2*. Las cajas orientadas (OBB), motivadas por [1], buscan reducir el fondo que un AABB holgado arrastra en lomos inclinados.

C. Segmentación de instancias y OCR

Mask R-CNN [7] extiende Faster R-CNN con una rama de máscara por instancia y RoIAlign, de modo que el lomo puede delimitarse a nivel de píxel y no solo mediante una caja. Esa representación importa cuando el AABB supera de forma notable al polígono y el OCR se alimenta del recorte. En

reconocimiento de texto, Shi *et al.* [8] introdujeron CRNN (CNN + RNN + CTC) como modelo extremo a extremo de secuencias; buena parte de los OCR prácticos usados en flujos de lomos [2] heredan esa idea de leer el recorte como secuencia. El presente trabajo se sitúa en esa intersección: compara AABB, OBB y máscaras, y evalúa OCR sobre los lomos localizados.

III. Problema y conjunto de datos

A. Problema de localización y lectura

A partir de una imagen de estantería se busca obtener lomos localizados y, en lo posible, texto recuperable por OCR. El problema impone dos exigencias acopladas. Por un lado, no omitir instancias en estantes densos ni en libros inclinados. Por otro, enviar al OCR un recorte donde predomine el lomo correcto, sin arrastrar de forma significativa al vecino ni al fondo. Un *mAP* de caja elevado no garantiza, por sí solo, un recorte legible.

B. Conjunto de datos Book Spine 2

Se utiliza *Book Spine 2* (Roboflow, espacio *bookspine-fxfsx*), versión **v4**, con exportación **YOLOv8**, etiquetas en **polígono** (una clase: lomo) y resolución 640×640. El particionado oficial se conserva tal cual: **1.265 / 271 / 120** imágenes (~76 / 16 / 7 %), con aproximadamente 20.354 / 4.622 / 1.747 lomos y densidad similar entre particiones (~15–17 lomos por imagen). La validación sirve para comparar modelos; la prueba se reserva como evaluación final sin ajustar hiperparámetros. El volumen alcanza para *fine-tuning*, aunque el conjunto de prueba es reducido en número de imágenes y su *mAP* resulta, en consecuencia, más variable.

C. Análisis exploratorio: variables medidas

Antes del entrenamiento se realizó un análisis exploratorio centrado en la **geometría de las etiquetas**, y no en color ni en proyecciones globales de la imagen. Histogramas RGB o incrustaciones tipo t-SNE mezclan fondo, condiciones de captura y del orden de 16 lomos por foto, de modo que no anticipan con claridad fallos de detección ni de recorte. Como YOLOv8 AABB y Faster R-CNN operan sobre **cajas alineadas a los ejes** derivadas del polígono, el análisis midió cinco familias de variables: ocupación (área del polígono sobre área del AABB), tamaño y aspecto del AABB, inclinación respecto de la vertical, solape entre AABB vecinos (máximo IoU por caja) y fuga entre particiones (*leakage*) por imagen fuente en el split de Roboflow (mismo nombre base, distinto sufijo *.rf.*<*hash*>).

D. Hallazgos del análisis exploratorio (validación, salvo indicación contraria)

El primer resultado es la holgura del AABB. Sobre 4.622 lomos de validación, la ocupación media es **0,54** (mediana 0,53; p10 = 0,18; p90 = 0,92): en promedio, el área de la caja casi duplica la del polígono, y esa holgura suele ser vecino o fondo. De ahí que no convenga alimentar el OCR con el AABB sin postprocesado, y que OBB y máscaras resulten representaciones más naturales para el recorte.

Las particiones son, además, homogéneas en tamaño —mediana cercana a 44–47 px de ancho y 350–380 px de alto, con razón alto/ancho mediana ~**7**—, pero el objeto es **alto y estrecho**. Aunque el criterio COCO [9] clasifique la mayoría como "grandes" por área, esa etiqueta engaña: el área relativa no captura la dificultad real. Un ~10 % de cajas son más anchas que altas, patrón típico de holgura o de filas.

La inclinación, por su parte, es bimodal. La mediana (~3,5–3,7°) describe lomos casi verticales, mientras que la media (~18°) y el p90 (~78°) revelan una cola pesada: cerca del **18 %** supera 45°. El dataset no es "todo inclinado"; conviven instancias paradas y fuertemente rotadas. Ese perfil justifica tanto el subconjunto *inclinado* como el brazo OBB.

En estanterías densas el solape es estructural. El máximo IoU mediano entre AABB vecinos es **0,27**; el **76 %** de las cajas solapa otra con IoU ≥ 0,1 y el **23 %** alcanza IoU ≥ 0,5. Un NMS agresivo en 0,5 puede eliminar verdaderos positivos. Por eso se define el subconjunto *denso* (imagen con al menos un GT cuyo máximo IoU con otro GT ≥ 0,5) y se adopta un NMS más permisivo en detección.

Finalmente, la fuga ocurre por archivo y no por escena original. El split de Roboflow opera a nivel de archivo: de 1.218 fuentes distintas, 104 cruzan particiones, de modo que el **27 %** de validación y el **33 %** de prueba tienen una imagen hermana en entrenamiento. Declararlo no invalida el experimento: el objetivo no es estimar el rendimiento absoluto sobre estanterías inéditas, sino comparar modelos bajo el mismo particionado. Con ese sesgo compartido, las diferencias de *mAP*, recall y latencia siguen siendo atribuibles a la representación y al entrenamiento; el *mAP* absoluto, en cambio, debe leerse con cautela fuera de este split.

E. Implicaciones para el diseño experimental

Esos hallazgos fijan el protocolo que sigue. Se mantiene *imgsz* en al menos 640; se reportan métricas globales y también en densos e inclinados; se comparan AABB, OBB y segmentación como respuestas distintas al mismo diagnóstico geométrico; y se trata el OCR como etapa sensible a la calidad del recorte —ajuste de caja, rotación o máscara—, no como lectura directa del AABB sin postprocesado.

IV. Metodología

A. A. Diseño experimental

El diseño sigue el diagnóstico de la Sección III. Se comparan seis modelos bajo tres representaciones —**AABB** (YOLOv8s, Faster R-CNN ResNet-50 FPN), **OBB** (YOLOv8s-OBB, YOLO26s-OBB) y **máscara de instancia** (YOLO26s-seg, Mask R-CNN ResNet-50 FPN)— conservando el particionado oficial de *Book Spine 2* v4, sin reasignar imágenes. La validación se usa para elegir *checkpoint* y comparar; la prueba es una pasada única sin ajustar umbrales. Como complemento, se reportan dos subconjuntos de validación definidos por geometría: *denso* (imagen con al menos un GT cuyo máximo IoU AABB con otro GT ≥ 0,5) e *inclinado* (GT con inclinación > 45°).

B. Formato de etiquetas y conversión a COCO

El dataset se descarga en exportación **YOLOv8** (etiquetas por imagen y *data.yaml*), con polígonos de instancia como representación canónica del lomo. No todos los modelos

consumen ese formato de forma nativa, así que el preprocesado se bifurca según el marco de entrenamiento.

En la familia YOLO (YOLOv8s, YOLOv8s-OBB, YOLO26s-OBB, YOLO26s-seg), Ultralytics lee el export de manera directa —AABB, OBB o segmentación, según el brazo—. Allí **no se convierte a COCO**: se evita una transformación innecesaria y se mantiene el flujo estándar del proveedor. En la familia R-CNN (Faster R-CNN y Mask R-CNN), *torchvision* espera anotaciones en estilo **COCO** (cajas y, en segmentación, polígonos o máscaras por imagen). Por eso se aplica una **conversión YOLO → COCO** a partir de los mismos polígonos y del mismo particionado, sin reetiquetar a mano ni cambiar la semántica de la clase *lomo*.

La decisión no es una preferencia arbitraria de representación, sino un requisito de entrada de cada biblioteca. Unificar todo a COCO habría forzado conversión en los brazos YOLO sin beneficio; omitirla en R-CNN habría sido incompatible con el *dataloader* de *torchvision*. En ambos caminos, la geometría de referencia permanece anclada a los polígonos originales de *Book Spine 2* v4.

C. Entrenamiento por familia

Todos los experimentos parten de pesos preentrenados en COCO y hacen *fine-tuning* a una clase (*lomo*). La resolución efectiva se mantiene en al menos 640 px (*imgsz* / *min_size*), en línea con el análisis exploratorio. Cuando el marco lo permite se fija semilla 42. El hardware de referencia es Google Colab con GPU Tesla T4. No se unifican las épocas entre brazos: cada familia elige el mejor *checkpoint* en validación (*mAP* / *fitness*), para no forzar un presupuesto fijo que sobreentrene a unos o truncue prematuramente a otros.

En Ultralytics se entrenan YOLOv8s (AABB), YOLO26s-seg y las variantes OBB sobre el export YOLO nativo (*data.yaml*). Los hiperparámetros típicos de AABB y segmentación son *imgsz* = 640, *batch* = 16, SGD (*lr0* = 0,01, *lrf* = 0,01, *momentum* = 0,937), NMS con *iou* = 0,40 —más permisivo que el valor por defecto ~0,7, motivado por el solape del EDA— y *max_det* = 300. Las corridas de referencia incluyen YOLOv8s-AABB (20 épocas; mejor *checkpoint* en la 19), YOLO26s-seg (hasta 30 épocas con *patience* = 5; mejor época 24 según *mAP* de máscara) y OBB (25 épocas, SGD con $lr0 = 1\times10^{-3}$ y *batch* = 32). El mejor modelo se elige por *fitness* Ultralytics / *mAP@0.5* de la salida principal.

En *torchvision* se entrenan Faster R-CNN y Mask R-CNN con ResNet-50 FPN (variante v2 en Faster) sobre las anotaciones ya convertidas a COCO. Se usan *min_size* = 640, *max_size* = 1333, *batch* = 2, SGD con *lr* = 0,005 y *momentum* = 0,9, y NMS de cajas 0,40 en Faster (alineado a YOLO). El *checkpoint* se selecciona por el mayor *mAP@0.5* en validación. En las corridas de referencia, Faster alcanza su mejor *mAP@0.5* en la época 6 —extender a 30 épocas no mejora esa métrica—, y Mask lo hace en la 12.

El resumen de las métricas de entrenamientos se observa en la Tabla I

TABLA I – MÉTRICAS ENTRENAMIENTOS

Mod.	Tarea	mAP@ 0.5	mAP@ 0.5:0.95	Prec.	Recall	Lat. (ms)
YOLOv8s	AABB (caja)	0.822	0.676	0.976	0.823	~14
Faster R-CNN	AABB (caja)	0.799	0.603	0.965	0.791	~119
YOLOv8s-OBB	OBB (caja orientada)	0.728	0.351	0.759	0.677	~31
YOLO26s-OBB	OBB (caja orientada)	0.848	0.532	0.835	0.761	~18
YOLO26s-seg	Máscara	0.780	0.562			~26
Mask R-CNN	Máscara	0.785	0.507	0.785	0.834	~117

AABB, OBB y Máscara no constituyen un ranking único: el mAP@0.5 no es estrictamente comparable entre tipos de tareas

D. Protocolo de evaluación

Las métricas reportadas son *mAP@0.5*, *mAP@0.5:0.95*, precisión, recall y latencia media (ms/imagen, *batch* = 1 en AABB y segmentación). En AABB y segmentación, al reevaluar el mejor *checkpoint* se fija *conf*/*score* ≥ 0,5 e *IoU* ≥ 0,5. Los *mAP* de caja AABB, OBB y máscara no se agregan en un *ranking* único: se tabulan por tipo de tarea y, de forma complementaria, en un resumen de seis filas con la advertencia de comparabilidad. Las cifras OBB corresponden a la validación Ultralytics del *best.pt* al cierre del entrenamiento; no se completó la misma pasada de prueba y subconjuntos que en AABB y segmentación porque el pipeline de OCR se cerró con YOLO26s-seg, y la homogeneización OBB quedó como trabajo futuro.

E. OCR sobre lomos localizados

El flujo de OCR no se alimenta del AABB sin postprocesado. El recorte se obtiene del polígono o de la máscara —o de la caja orientada cuando corresponde—, se alinea el texto por rotación y se preprocesa antes de la lectura con motores de uso práctico (EasyOCR y PaddleOCR), en un esquema cercano a pipelines de lomos basados en YOLO y PaddleOCR [2]. Los umbrales de detección se alinean al resto del experimento (*conf* = 0,5, NMS *iou* = 0,40). La experimentación, la comparación entre motores y el indicador de coincidencia contra un catálogo local se desarrollan en la Sección VII; el prototipo de demostración, en la Sección VIII. El matching many-to-many a escala permanece como trabajo futuro (Sección IX).

V. EXPERIMENTOS Y RESULTADOS

A. Detección AABB

En AABB, YOLOv8s y Faster R-CNN comparten un perfil de alta precisión y recall más contenido. En validación, YOLOv8s alcanza *mAP@0.5* = 0,822 y Faster R-CNN 0,799, con precisiones cercanas a 0,97 y recalls en el entorno de 0,79–0,82. La diferencia aparece sobre todo en latencia: ~14 ms por imagen en YOLO frente a ~119 ms en Faster. En prueba, Faster R-CNN alcanza 0,829 en *mAP@0.5*; dado el tamaño reducido de esa partición, ese valor no se usa para elegir modelo. En densos e inclinados ambos degradan, y la limitación dominante sigue siendo el recall, coherente con el solape entre cajas y con el NMS.

B. Detección OBB

Dentro del brazo OBB, y bajo el protocolo Ultralytics al cierre del entrenamiento, YOLO26s-OBB (*mAP@0.5* = 0,848) supera a YOLOv8s-OBB (0,728) y además es más rápido (~18 ms frente a ~31 ms). El *mAP@0.5:0.95* queda por debajo del de AABB (0,53 frente a ~0,60–0,68), resultado esperable al exigir ajuste angular. Estas cifras no incluyen reevaluación con *conf* ≥ 0,5 ni prueba/subconjuntos —el pipeline de OCR se cerró con YOLO26s-seg—; se interpretan solo dentro del brazo OBB.

C. Segmentación de instancias

En máscara, YOLO26s-seg y Mask R-CNN quedan muy próximos en *mAP@0.5* de validación (0,780 frente a 0,785) y en prueba (~0,776 en ambos). Mask R-CNN aporta más recall (0,834) a costa de ~117 ms por imagen; YOLO26s-seg opera cerca de 26 ms.

En densos e inclinados el *mAP@0.5* desciende a ~0,58–0,65: Mask conserva mejor recall y YOLO mayor precisión. Cuando se miden ambas salidas, la brecha entre caja y máscara confirma que localizar el AABB no equivale a segmentar el polígono.

TABLA II — SEGMENTACIÓN DE INSTANCIAS: VALIDACIÓN / PRUEBA / SUBCONJUNTOS DENSO E INCLINADO

Modelo	Corte	mAP@0.5	mAP@0.5:0.95	Precision	Recall
YOLO26s-seg	Val	0.780	0.562		
YOLO26s-seg	Test	0.776	0.558		
YOLO26s-seg	Val todas (subconj.)	0.735	0.545	0.958	0.743
YOLO26s-seg	Denso	0.584	0.413	0.925	0.594
YOLO26s-seg	Inclinado	0.620	0.437	0.962	0.627
Mask R-CNN	Val	0.785	0.507	0.785	0.834
Mask R-CNN	Test	0.776	0.518	0.757	0.839
Mask R-CNN	Denso	0.648	0.341	0.661	0.728
Mask R-CNN	Inclinado	0.647	0.335	0.657	0.746

(1) COLUMNA *MAP* = MÉTRICA DE MÁSCARA (*CONF*/*SCORE* ≥ 0,5, *IOU* ≥ 0,5).
(2) *DENSO*: AL MENOS UN GT CUYO MÁXIMO IOU AABB CON OTRO GT ≥ 0,5.
(3) *INCLINADO*: GT CON INCLINACIÓN > 45°.

D. Remisión a OCR

La evaluación del texto recuperado sobre los lomos localizados —con YOLO26s-seg como localizador— se presenta en la Sección VII.

VI. DISCUSIÓN

Los errores observados encajan con el análisis exploratorio. La ocupación AABB media de ~0,54 anticipa fondo o vecino en el recorte; el solape denso explica caídas de recall bajo NMS; la cola de inclinación justifica el brazo OBB sin transformar el dataset en un conjunto "solo rotado". En velocidad, las variantes YOLO superan a las R-CNN en un factor aproximado de 5–8×, con *mAP@0.5* comparable por tipo de tarea. Mask R-CNN no aventaja de forma consistente a YOLO26s-seg en *mAP* de máscara global, aunque su mayor recall en densos e inclinados puede preferirse cuando el objetivo es un inventario exhaustivo antes que tiempo real.

Hay, no obstante, límites que condicionan la lectura de las cifras. La fuga del 27–33 % por imagen fuente entre particiones puede volver optimista el *mAP* absoluto, aunque la comparación relativa sobre el mismo split sigue siendo útil. La prueba, con 120 imágenes, introduce variabilidad. El protocolo OBB no se homogeneizó con el de AABB y segmentación porque el pipeline de OCR se cerró con YOLO26s-seg. El OCR, por su parte, se evalúa sobre un conjunto reducido de fotos propias (Sección VII) y no sustituye un protocolo de inventario a escala de catálogo [3].

A la luz de ese balance se adopta **YOLO26s-seg** como localizador para el flujo de OCR. La máscara mitiga la holgura del AABB al delimitar el lomo a nivel de píxel [7] y reducir fondo o vecino en el recorte. Su *mAP@0.5* de máscara en validación (0,780) es comparable al de Mask R-CNN (0,785), pero con latencia mucho menor (~26 ms frente a ~117 ms). Frente a los detectores AABB, ofrece un recorte más ajustado sin abandonar el compromiso calidad–velocidad de la familia YOLO. Mask R-CNN puede conservar más recall en cortes difíciles, y YOLO26s-OBB alcanza un *mAP@0.5* elevado bajo su protocolo propio; aun así, YOLO26s-seg concentra en un solo modelo la representación geométrica adecuada para OCR, una calidad de máscara competitiva y una latencia compatible con un pipeline de inventario visual.

VII. EXPERIMENTACIÓN CON OCR

Una vez elegido YOLO26s-seg como localizador, resta comprobar si el recorte que produce es usable para lectura. El experimento se realiza sobre dos fotografías propias de estantería, fuera de *Book Spine 2*, de modo que el pipeline se pone a prueba en condiciones cercanas al uso previsto.

A. Protocolo experimental

Sobre cada imagen se corre el *checkpoint* de YOLO26s-seg con los mismos umbrales del resto del trabajo (*conf* = 0,5, NMS *iou* = 0,40). De cada máscara se obtiene un recorte alineado al eje del lomo —no el AABB sin postprocesado— y se aplica un preprocesado ligero antes de la lectura. Sobre esos recortes se evalúan dos motores de OCR de uso práctico, EasyOCR (español e inglés) y PaddleOCR (español), midiendo texto recuperado, confianza y latencia por instancia. La inclusión de PaddleOCR permite contrastar, en un escenario acotado, un motor ya empleado en pipelines de lomos [2] frente a una alternativa de uso frecuente.

Como indicador de utilidad para inventario, cada lectura se compara de forma difusa contra un catálogo local de títulos mediante *token set ratio* (RapidFuzz). Se declara coincidencia cuando el puntaje alcanza un umbral de 85/100. El emparejamiento no pretende resolver el matching many-to-many de colecciones a gran escala [3]: solo mide si lo leído se acerca lo suficiente a un título real como para identificarlo.

B. Resultados

La detección produjo **19** recortes (5 en la primera foto y 14 en la segunda). PaddleOCR supera con claridad a EasyOCR en las tres dimensiones observadas. Su confianza media es 0,95 frente a 0,57; la latencia mediana por recorte es del orden de 0,6 s frente a ~38 s; y, en el matching contra catálogo, recupera **8/19** coincidencias (42,1 %) frente a **3/19** (15,8 %).

Los fallos residuales se concentran donde el recorte sigue siendo difícil de leer: lomos muy densos, tipografías pequeñas o interferencia de vecinos. En esos casos ninguno de los dos motores alcanza el umbral, lo que refuerza que la calidad del recorte condiciona al OCR tanto como la elección del motor.

C. Conclusiones de OCR

En este escenario acotado, el pipeline YOLO26s-seg → recorte por máscara → PaddleOCR es el más consistente: mejor texto, mayor tasa de identificación en catálogo y latencia compatible con un uso interactivo. EasyOCR, con los mismos recortes, degrada tanto la legibilidad como el tiempo de respuesta. El resultado confirma la premisa del EDA y de la Sección VI: un localizador que reduzca holgura ayuda al OCR, pero no lo vuelve trivial; densos e inclinados siguen limitando la lectura. Ampliar el experimento a más fotos y a un matching de catálogo más robusto queda como trabajo futuro.

VIII. Prototipo de demostración

Con esos resultados se implementó un prototipo del pipeline completo, pensado como demostración del trabajo y no como producto final. El flujo integra **YOLO26s-seg** para localizar lomos y **PaddleOCR** para leer el texto de cada recorte alineado, de modo que una fotografía de estantería pueda recorrerse extremo a extremo: detección, segmentación, recorte y lectura, con contraste opcional contra un catálogo local. La composición es deliberadamente próxima a pipelines de detección más OCR sobre lomos [2], aunque aquí el énfasis está en la demostración de las decisiones experimentales propias y no en proponer una arquitectura nueva.

El prototipo sirve para exhibir, en condiciones controladas, la elección del localizador y del motor de OCR, y para inspeccionar de forma cualitativa dónde el flujo acierta o falla (densidad, inclinación, tipografía). Su alcance es demostrativo: valida la viabilidad del pipeline elegido y facilita la comunicación de resultados, sin pretender cerrar el problema de inventario a escala [3].

IX. Conclusiones y trabajo futuro

Este trabajo partió de un diagnóstico geométrico de *Book Spine 2* v4 —AABB holgados, estantes densos e inclinación bimodal— y evaluó seis modelos en AABB, OBB y máscara de instancia bajo un protocolo común por tipo de tarea. No emerge un ganador absoluto: las variantes YOLO concentran el mejor compromiso entre calidad y latencia; OBB y máscaras responden de forma distinta al mismo problema de holgura; y densos e inclinados permanecen como el principal punto débil en recall.

Ese balance justifica adoptar **YOLO26s-seg** como localizador para lectura: máscara competitiva frente a Mask R-CNN con una fracción de su costo temporal, y un recorte más útil para OCR que el AABB crudo. Sobre fotos propias, **PaddleOCR** superó a EasyOCR en confianza, latencia e identificación difusa contra un catálogo local, sin eliminar los fallos ligados a densidad y tipografía. Con ese pipeline se implementó además un **prototipo de demostración** extremo a extremo, suficiente para validar la viabilidad del flujo y comunicar los hallazgos, aunque lejos aún de un sistema de inventario a escala.

Como trabajo futuro se propone, en primer lugar, homogeneizar la evaluación OBB con el mismo protocolo de prueba y subconjuntos usado en AABB y segmentación. En segundo lugar, ampliar el experimento de OCR a más imágenes y condiciones de captura, y madurar el prototipo —umbrales, NMS adaptativo a densidad y robustez del recorte— hacia un uso menos controlado. Por último, si el objetivo pasa del indicador local de coincidencia al inventario real, el matching many-to-many contra catálogos grandes debe tratarse como etapa propia, independiente del *mAP* de detección [3].

Referencias

[1] H. Ma, C. Wang, A. Li, A. Xu y D. Han, "An Accurate Book Spine Detection Network Based on Improved Oriented R-CNN," *Sensors*, vol. 24, no. 24, art. 7996, 2024, doi: 10.3390/s24247996.

[2] Z. Li, B. Guo y D. Mu, "Recognizing Text on Book Spines with Improved YOLOv11 and Optimized PaddleOCR," *Electronics*, vol. 14, no. 23, art. 4689, 2025, doi: 10.3390/electronics14234689.

[3] A. Llabrés, A. U. Dey, D. Karatzas y E. Valveny, "Image-text matching for large-scale book collections," arXiv:2407.19812, 2024.

[4] S. Ren, K. He, R. Girshick y J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," arXiv:1506.01497, 2015.

[5] J. Terven y D. M. Córdova-Esparza, "A Comprehensive Review of YOLO Architectures in Object Detection: From YOLOv1 to YOLOv8 and YOLO-NAS," arXiv:2304.00501, 2023.

[6] M. Yaseen, "What is YOLOv8: An In-Depth Exploration of the Internal Features of the Next-Generation Object Detector," arXiv:2408.15857, 2024.

[7] K. He, G. Gkioxari, P. Dollár y R. Girshick, "Mask R-CNN," en *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, arXiv:1703.06870.

[8] B. Shi, X. Bai y C. Yao, "An End-to-End Trainable Neural Network for Image-Based Sequence Recognition and Its Application to Scene Text Recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, 2017, arXiv:1507.05717.

[9] T.-Y. Lin *et al.*, "Microsoft COCO: Common Objects in Context," en *ECCV*, 2014, arXiv:1405.0312.